Data Quality ETL Pipeline

Production-oriented batch framework for validation, transformation, reliable loading, and Power BI reporting

Orchestration: Apache Airflow

Processing: Apache Spark

Storage: MinIO data lake (Bronze / Silver / Gold)

Reporting: Power BI

Deployment: Docker Compose and optional GitHub Actions

Project goal

Detect invalid data before it reaches business users, isolate rejected records, and produce trusted curated datasets for analysis. This version excludes Prometheus, Grafana, Alertmanager, and related monitoring/observability components.

Project Documentation — Parts 1–3

Project: Data Quality ETL Pipeline using Apache Airflow, Apache Spark, MinIO, and Power BI

This section covers project planning and management, a review of comparable tools/solutions, and functional/non-functional requirements. Team names and exact calendar dates can be filled in by the team before submission.

1. Project Planning & Management

1.1 Project idea

The project builds a batch ETL pipeline that ingests data from REST APIs and file-based sources, stores raw data in MinIO, validates data quality, transforms valid records with Apache Spark, separates rejected records, and publishes curated Bronze/Silver/Gold datasets for reporting in Power BI. Apache Airflow coordinates the workflow and supports task retries and controlled reruns.

1.2 Objectives

- Build a repeatable, scheduled batch ETL workflow orchestrated by Airflow.
- Implement quality checks for schema, nulls, duplicates, row counts, referential integrity, and value ranges.
- Keep raw inputs for traceability and reprocessing; isolate rejected records with clear reasons.
- Transform valid records with Spark and store curated datasets in MinIO.
- Provide Power BI reports for processed/rejected volumes and quality outcomes.
- Demonstrate failure handling, retry behavior, and recovery through a controlled test.

1.3 Scope

In scope	Out of scope / limitations
Batch ingestion from REST API and CSV/JSON; optional operational DB source.	Real-time streaming and sub-second alerting.
MinIO Bronze/Silver/Gold data lake layers.	A separate PostgreSQL data warehouse.
Spark transformations and reusable validation rules.	Prometheus, Grafana, Alertmanager, or a dedicated observability stack.
Airflow DAG orchestration, retries, and safe reruns.	Production high-availability deployment unless separately added.
Power BI reporting from a compatible data access/refresh path.	Assuming Power BI can query MinIO directly without a supported connector/integration.

1.4 Proposed implementation schedule

Week	Main activities	Deliverables
1	Confirm requirements, data sources, sample data, architecture, and success criteria.	Approved scope, architecture, task plan.
2	Set up Docker Compose, Airflow, Spark, MinIO, buckets, and connectivity.	Runnable development environment.
3	Implement ingestion and Bronze storage; define schemas and data contracts.	Ingestion DAG and Bronze datasets.
4	Implement quality checks, rejected-record handling, Spark transformations, Silver/Gold outputs.	Validation module and curated datasets.
5	Prepare Power BI model/report; implement retries and rerun/recovery test cases.	Report prototype and recovery demo.
6	Run integration tests, fix issues, document setup/results, rehearse final demo.	Final documentation, tested demo, repository.

The schedule is a proposed six-week plan and should be aligned with the course deadline and actual team availability.

1.5 Team roles and responsibilities (five-person example)

Role	Responsibilities	Expected output
Member 1 — Project coordinator	Track milestones, scope, meetings, risks, and final integration.	Plan, task board, final integration checklist.
Member 2 — Ingestion & storage	API/file ingestion, MinIO buckets, Bronze/Silver/Gold layout.	Working ingestion and storage paths.
Member 3 — Data quality	Schema/null/duplicate/range/row-count/reference rules; rejected rows.	Validation module and test cases.
Member 4 — Processing & orchestration	Spark transformations, Airflow DAG, retries and rerun behavior.	Working DAG and transformation jobs.
Member 5 — Reporting & QA	Power BI report, integration tests, documentation and demo materials.	Report, test evidence, user guide.

Roles are a suggested division, not assigned to named people. For a smaller/larger team, combine or split responsibilities; all members should review integration and documentation.

1.6 Risks and mitigation

Risk	Impact	Mitigation
Source/API unavailable or rate-limited	Ingestion fails or is incomplete.	Use timeouts, bounded retries, sample fixtures, and record source/run context.
Unexpected schema or malformed records	Transformations fail or wrong results are produced.	Validate schema early; quarantine invalid rows; version data contracts.
Duplicate processing after rerun	Duplicate rows or inconsistent outputs.	Use deterministic run/output paths, idempotent writes, and deduplication keys.
MinIO permissions or endpoint misconfiguration	Read/write failures.	Test bucket access early; use least-privilege credentials and environment variables.
Power BI connectivity/refresh limitations	Reports cannot load or refresh data as expected.	Prototype the connector early; validate supported formats and refresh path; document fallback options.
Schedule/team dependency slips	Late integration or incomplete deliverables.	Set weekly milestones, track blockers, reserve final week for integration and fixes.

1.7 Success criteria / KPIs

KPI	Target for project demonstration
Pipeline execution	At least one full successful end-to-end batch run.
Quality coverage	All six planned rule categories (schema, null, duplicates, row count, referential integrity, ranges) tested.
Rejected data	Invalid rows isolated with run ID and failure reason.
Recovery	One transient failure demonstrates retry; one invalid input is corrected and reprocessed.
Storage outputs	Expected Bronze/Silver/Gold outputs exist in MinIO and can be read back.
Reporting	Power BI displays batch volumes and quality outcomes after a verified refresh.
Testing/documentation	DAG import and core tests pass; setup and limitations are documented.

2. Literature Review

This review compares established tools and approaches relevant to the project. The goal is not to claim that one tool replaces the entire platform, but to explain which ideas are adopted and where the proposed solution differs.

2.1 Comparable tools and solutions

Solution / source	Strengths	Limitations for this project	How our project uses the idea
Apache Airflow — official DAG/task documentation [1]	Defines workflow dependencies, task execution, scheduling, and retries.	Orchestration alone does not define the project's business-specific quality rules or lake layout.	Airflow coordinates ingestion, validation, transformation, and loading; task retries support recovery.
Great Expectations — validation documentation [2]	Declarative expectations, validation results, and reusable checks; supports documenting expected data behavior.	Adds a framework and configuration to learn/maintain; project may need Spark-specific integration choices.	We adopt the same validation mindset but implement focused checks in Spark/Python to match project scope; GX could be evaluated later.
MinIO — S3 compatibility [3]	Object storage organized through S3-compatible APIs; suitable for data-lake files and application integration.	Object storage is not itself a SQL query engine; analytics tools need a compatible connector or another query/access layer.	MinIO stores Bronze/Silver/Gold and rejected files; connector and refresh feasibility is a project risk to test.
Power BI — data connection documentation [4]	Connects to supported data sources and supports configured refresh/gateway workflows.	Only documented/supported connectors and capabilities should be assumed; direct MinIO querying is not guaranteed by default.	Power BI presents curated and quality-result data after the team validates the access path and refresh process.

2.2 Comparison with the proposed project

Dimension	Typical tool capability	Our proposed approach
Orchestration	Airflow defines DAG tasks and dependencies; retries can be configured.	Use a DAG with explicit ingestion → quality gates → transformation → MinIO writes.
Quality validation	Great Expectations provides reusable expectation suites and detailed validation outcomes.	Implement required rule categories directly; evaluate a framework later if rule complexity grows.
Storage	MinIO exposes S3-compatible object storage for object/bucket operations.	Use data-lake prefixes for Bronze, Silver, Gold, rejected records, and batch result files.
Reporting	Power BI connects through supported connectors and data-access paths.	Use Power BI for batch/quality reporting only after connector and refresh tests.
Scope	Individual tools solve different parts of the data lifecycle.	Integrate the tools into one small, demonstrable batch ETL workflow with recovery.

2.3 Lessons learned from the review

- Separate responsibilities: Airflow orchestrates; Spark processes; MinIO stores objects; Power BI reports.
- Make quality expectations explicit and measurable rather than relying on visual inspection.
- Keep rejected rows separate from trusted outputs, with enough metadata to explain each rejection.
- Design retries together with idempotent writes so recovery does not create duplicates.
- Verify Power BI connectivity and scheduled refresh early; do not assume every S3-compatible store is natively queryable by every connector.

2.4 References

[1] Apache Airflow, “DAGs” and “Tasks” documentation. https://airflow.apache.org/docs/apache-airflow/stable/concepts/dags.html and https://airflow.apache.org/docs/apache-airflow/stable/concepts/tasks.html

[2] Great Expectations, “Run Validations” documentation. https://docs.greatexpectations.io/docs/core/run_validations/

[3] MinIO, “S3 Compatible Storage.” https://min.io/product/s3-compatibility

[4] Microsoft Learn, “Power BI data connection documentation” and “Power BI data sources.” https://learn.microsoft.com/en-us/power-bi/connect-data/ and https://learn.microsoft.com/en-us/power-bi/connect-data/power-bi-data-sources

3. Requirements Gathering

3.1 Stakeholders and needs

Stakeholder	Needs / expectations
Business users / analysts	Trustworthy curated data, understandable Power BI reports, clear definitions for rejected and accepted rows.
Data engineer	Repeatable ingestion, schema control, scalable Spark transformations, traceability, and safe reruns.
Project team / maintainer	Clear setup instructions, modular code, tests, version control, and understandable error messages.
Project evaluator / supervisor	Demonstrable requirements, documented architecture, measurable acceptance criteria, and reproducible test evidence.
Data owner / source-system owner	Respect source constraints, handle schema changes, avoid unauthorized exposure of sensitive data.

3.2 Functional requirements

ID	Requirement	Priority
FR-01	The system shall ingest batch data from REST API and CSV/JSON sources; additional sources are optional.	Must
FR-02	The system shall preserve raw input in the MinIO Bronze layer for traceability and reprocessing.	Must
FR-03	The system shall validate schema and required data types before trusted outputs are produced.	Must
FR-04	The system shall detect null violations and duplicates according to configured rules.	Must
FR-05	The system shall validate row-count bounds, referential integrity, and numeric/date ranges where applicable.	Must
FR-06	The system shall isolate rejected records and include a rejection reason and run identifier.	Must
FR-07	The system shall transform valid records with Spark and write curated Silver/Gold datasets to MinIO.	Must
FR-08	The system shall orchestrate tasks through an Airflow DAG with dependencies and configured retries.	Must
FR-09	The system shall allow a failed or corrected batch stage to be rerun without duplicating trusted outputs.	Must
FR-10	The system shall provide Power BI reporting for accepted/rejected volumes and quality results through a verified connector/refresh path.	Must
FR-11	The system should run in a reproducible Docker Compose development environment.	Should
FR-12	The project may use GitHub Actions for linting, tests, DAG checks, and Compose validation.	Could

3.3 Non-functional requirements

Category	Requirement / measurable expectation
Performance	For the selected demo dataset, complete a full batch within the time agreed by the team; record runtime and row counts to establish a baseline.
Reliability	Transient failures should use bounded retries; permanent failures should stop the affected run and preserve diagnostic details.
Data integrity	Only records that pass applicable checks are written to trusted curated outputs; rejected records remain separately identifiable.
Security	Keep credentials in environment variables/secrets, restrict MinIO bucket access, use least privilege, and avoid publishing sensitive row payloads in Power BI.
Traceability	Each batch should have a run identifier and recorded status/counts; rejected records should reference the run and failed rule.
Maintainability	Separate ingestion, quality, transformation, storage, and reporting code into modules with reusable functions.
Usability	Power BI pages should use clear titles, consistent KPI definitions, readable visuals, and filters/drill-through where useful.
Portability	Document prerequisites and provide Docker Compose configuration for reproducible local setup.
Testability	Quality rules should be tested with both valid and intentionally invalid examples; pipeline recovery should be demonstrated.

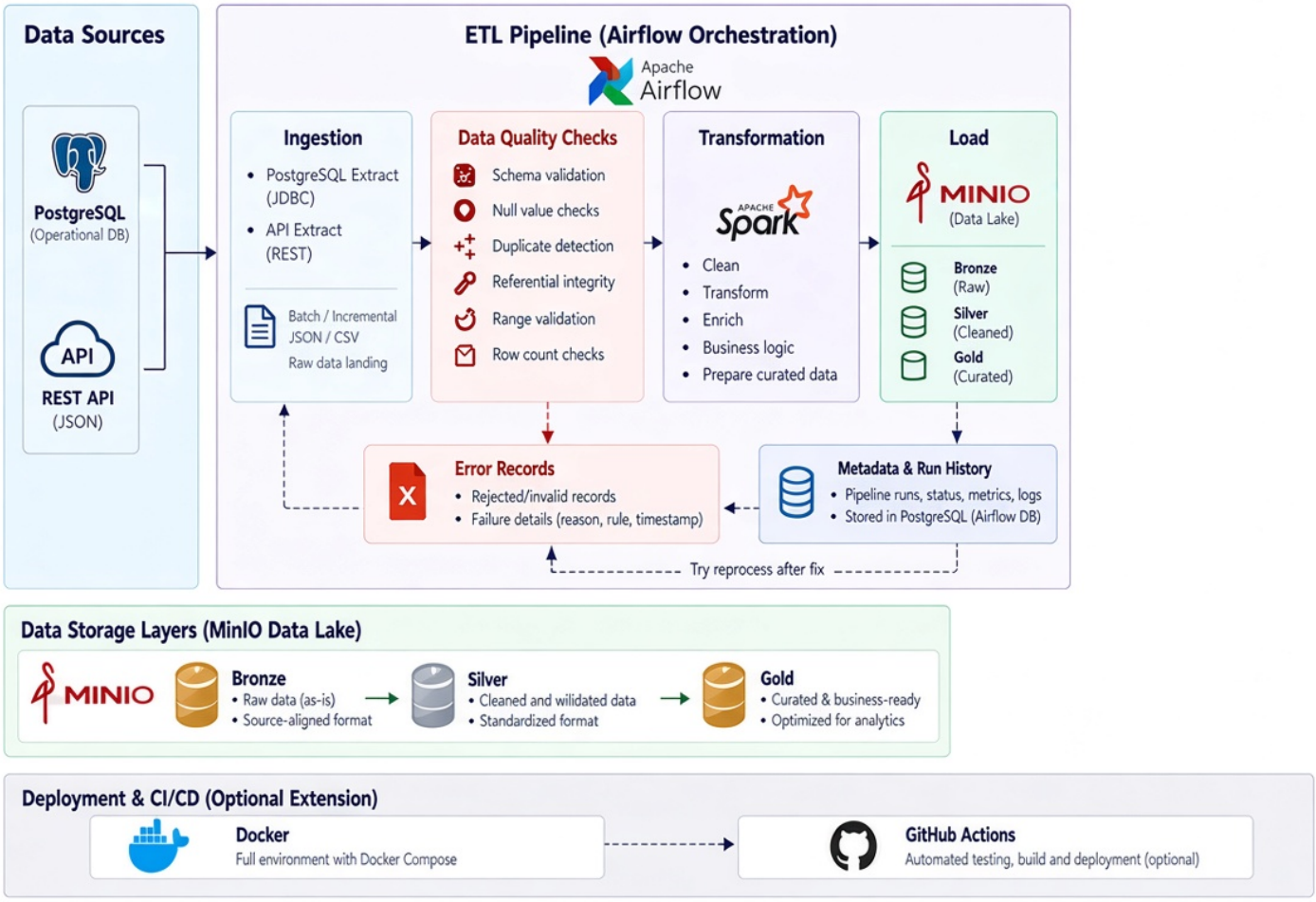
3.4 Data and interface requirements

- Inputs: REST API JSON responses and/or CSV/JSON files; document required fields, types, and expected update frequency.
- Storage: MinIO buckets/prefixes for bronze, silver, gold, rejected, and optional quality-result files.
- Processing: Spark batch jobs for validation, cleaning, enrichment, and writing curated outputs.
- Orchestration: Airflow DAG tasks with explicit dependencies, retries, and documented failure behavior.
- Reporting: Power BI consumes curated and quality-result data through a supported connector or integration; the chosen route must be tested.
- Outputs: curated datasets, rejected-record datasets, and a final report that summarizes batch and data-quality results.

3.5 Acceptance tests

Test ID	Scenario	Expected result
AT-01	Run a valid sample batch.	Airflow DAG succeeds; expected MinIO outputs are created.
AT-02	Provide a row with a required null.	Rule fails for the row; it is rejected or the task fails according to policy.
AT-03	Provide duplicate business keys.	Duplicates are detected and handled according to the configured rule.
AT-04	Provide schema/type mismatch.	Validation blocks the affected dataset before trusted output.
AT-05	Simulate temporary storage/source failure.	Airflow retry occurs within the configured limit.
AT-06	Correct invalid input and rerun.	Valid output is produced without duplicate trusted records.
AT-07	Refresh Power BI after a completed batch.	Report displays the latest available curated/quality results.

Requirements note: Numerical performance targets should be set after the team confirms dataset size and available hardware. The MinIO-to-Power BI connector path is a dependency that must be verified experimentally before claiming direct connectivity.


Data Sources
PostgreSQL
(Operational DB)
API
REST API
(JSON)
ETL Pipeline (Airflow Orchestration)
Apache Airflow
Ingestion
PostgreSQL Extract (JDBC)
API Extract (REST)
Batch / Incremental
JSON / CSV
Raw data landing
Data Quality Checks
Schema validation
Null value checks
Duplicate detection
Referential integrity
Range validation
Row count checks
Transformation
Apache Spark
Clean
Transform
Enrich
Business logic
Prepare curated data
Load
MINIO
(Data Lake)
Bronze
(Raw)
Silver
(Cleaned)
Gold
(Curated)
Error Records
Rejected/invalid records
Failure details (reason, rule, timestamp)
Metadata & Run History
Pipeline runs, status, metrics, logs
Stored in PostgreSQL (Airflow DB)
Try reprocess after fix

Data Storage Layers (MinIO Data Lake)
MINIO
Bronze
Raw data (as-is)
Source-aligned format
Silver
Cleaned and wilidated data
Standardized format
Gold
Curated & business-ready
Optimized for analytics

Deployment & CI/CD (Optional Extension)
Docker
Full environment with Docker Compose
GitHub Actions
Automated testing, build and deployment (optional)

3. End-to-End Pipeline Flow

Stage	Activity
1. Trigger	Airflow starts the DAG on schedule or through a manual/external trigger.
2. Ingestion	Read from REST APIs, CSV/JSON files, or configured source systems.
3. Bronze	Write source data to MinIO in original or minimally changed form.
4. Quality checks	Validate schema, required fields, duplicates, row counts, references, and ranges.
5. Rejected records	Write invalid rows to a separate MinIO prefix with rule and reason.
6. Transformation	Spark cleans, standardizes, enriches, and prepares valid data.
7. Silver / Gold	Write cleaned and curated Parquet/Delta datasets to MinIO.
8. Reporting	Power BI reads compatible curated and quality-result datasets.

Suggested DAG

start → extract → write_bronze → validate_schema → validate_nulls → validate_duplicates → validate_referential_integrity → validate_ranges → transform → write_silver_gold → end

Example MinIO layout

```
s3://data-lake/bronze/<source>/...
s3://data-lake/silver/<dataset>/...
s3://data-lake/gold/<dataset>/...
s3://data-lake/rejected/<pipeline>/run_id=.../...
```

4. Data Quality Framework

Check	What it validates	Typical action
Null values	Required fields contain no unexpected NULLs.	Reject rows or fail the task.
Duplicates	Business/technical keys are unique where required.	Quarantine duplicates.
Schema/types	Columns and types match the expected contract.	Fail before transformation.
Row count	Volume is within expected bounds.	Warn or fail on abnormal volume.
Referential integrity	Foreign keys match valid parent records.	Reject orphan rows.
Range validation	Numeric/date values are within business limits.	Reject invalid rows.

Quality result fields

For each rule, retain: run_id, pipeline_name, task_name, check_name, check_status, records_checked, records_failed, threshold, execution_timestamp, and error_message. Store quality results as structured JSON/Parquet files in MinIO when they are needed for Power BI reporting.

5. Reliability, Error Handling & Recovery

Mechanism	Implementation
Exception handling	Capture errors during extraction, validation, transformation, and writes.
Retries	Use Airflow retries and a delay for transient source or storage failures.
Failed-task handling	Record useful task/run context in Airflow logs and task state.
Rejected records	Save source, run ID, failed rule, reason, and timestamp to MinIO.
Recovery	Rerun safely using deterministic output paths or idempotent writes.

Failure-recovery demonstration

- Inject an invalid row or simulate a temporary source/storage failure.
- Inspect the failed Airflow task and its error details.
- Verify invalid rows are isolated in the MinIO rejected prefix.
- Demonstrate retry behavior for a transient failure.

- Correct the input and rerun the affected stage.
- Confirm valid data reaches the curated MinIO layer and appears in Power BI after refresh.

6. Power BI Batch Reporting

Power BI is used for business-facing batch reports and analysis of curated datasets and data-quality outcomes. It is not used as a pipeline orchestration or real-time monitoring service.

Report page	Suggested content
Batch Overview	Processed rows, accepted rows, rejected rows, rejection percentage.
Data Quality	Failures by rule, null violations, duplicate counts, range violations.
Pipeline Results	Batch completion outcome, successful/failed task counts where available.
Volume Trends	Row-count trends and deviations from expected volume.
Rejected Data	Rejection reasons and permitted record-level drill-through.

Connectivity note

Confirm the chosen MinIO-to-Power BI connector supports the S3-compatible endpoint and the stored file format. If direct file access is unsupported, an additional query layer may be needed; it is intentionally not included in this design.

7. Deployment, Testing & Project Structure

Component	Role
Airflow	Scheduling, DAG execution, dependencies, and retries.
Spark	Batch processing, quality checks, and transformation.
MinIO	Object storage for raw, curated, and rejected datasets.
Power BI	External reporting client connected through a compatible integration.
Docker Compose	Local deployment of the core services.
GitHub Actions (optional)	Linting, unit tests, DAG checks, and Compose validation.

Suggested repository structure

```
project/
  dags/ # Airflow DAGs
  src/ingestion/ # Extraction and Bronze writes
  src/quality/ # Validation rules
  src/transformations/ # Spark transformations
  src/storage/ # MinIO read/write helpers
  tests/ # Unit and integration tests
  powerbi/ # Report/model documentation
  data/sample/ # Safe sample inputs
  docker-compose.yml
  README.md
```

Testing checklist

- Unit-test quality rules with valid and invalid samples.
- Validate Airflow DAG imports.
- Validate Docker Compose configuration.
- Test MinIO write/read and access permissions.
- Validate Power BI connectivity and refresh with a small curated dataset.

8. Definition of Done

#	Acceptance criterion
1	Airflow executes the representative batch pipeline.
2	All required quality checks return measurable results.

#	Acceptance criterion
3	Invalid records are isolated in MinIO rejected storage.
4	Transient failures demonstrate retry behavior.
5	Errors have useful task-level diagnostic details.
6	Bronze, Silver, and Gold datasets are written to MinIO.
7	Power BI displays batch results and data-quality reports.
8	Recovery is demonstrated by correcting input and rerunning.
9	Docker Compose starts the core environment; CI checks are optional.

Summary

This project demonstrates batch data engineering practices: quality gates, rejected-record isolation, retry handling, MinIO-based lake storage, Spark transformations, and Power BI reporting. Prometheus, Grafana, Alertmanager, PostgreSQL data warehouse components, and related observability content have been removed.